

# Dokumentasjon - Prosjektoppgave Objektorientert programmering

Albert Lindberg, Clemens Beese

April 2026

## 1 Beskrivelse av appen

*Her skal dere gi en kort beskrivelse av appen dere har laget og hva den gjør. (krav: 150-200 ord)*

Appen vi har valgt å lage er ment å fungere som en "Game Launcher" (eks. Steam), og et par forskjellige spill. Det skal altså være et brukergrensesnitt som lar deg navigere mellom de forskjellige spillene og tilbake til hovedmenyen, se på highscores, endre på innstillinger, o.l.

Grensesnittet er splittet i et par deler for å enkelt kunne gjenbruke det i de forskjellige spillene. Vi har en hovedmeny som tar seg av "Game Launcher" funksjonaliteten, en highscore-meny, en pausemeny, en gameover-meny, og en enkel instilling-meny som lar deg endre navn. I tillegg har vi de spillspesifikke grensesnittene. Ettersom f.eks. pausemenyen er generell, kan pauselogikken deles mellom alle spillene, og det samme gjelder gameover.

Tanken var at å lage spill var en morsom/interessant måte å kunne bruke pensum i praksis, og siden vi valgte å samarbeide ønsket vi å lage flere forskjellige spill slik at vi kunne prøve mye forskjellig. Som et tidsbesparende tiltak brukte vi KI tjenesten Claude til å gi grensesnittet et fint design, slik at vi kunne fokusere på funksjonaliteten til grensesnittet og spillene.

## 2 Diagram

*Her skal dere vise et bilde av et diagram. Dere kan selv velge blant de fire diagramtypene i pensum; klassediagram, objektdiagram, objekttilstandsdiagram, og sekvensdiagram. Diagrammet skal vise en **interessant/viktig** del av appen. Dersom dere gjør noen antagelser som er viktige for å forstå diagrammet deres, skal disse også noteres ned her. (krav: Kun ett diagram)*

Klassediagrammet under viser forholdet mellom den abstrakte klassen `GameController` og de ulike `controller`-klassene til hvert enkelt spill. I tillegg viser den splitten mellom `controller`-klassene som håndterer koblingen til JavaFX, og `gameloop`-klassene som brukes for å oppdatere spillene med sin interne logikk. `GameLoop`-klassene er subclasser av JavaFX klassen `AnimationTimer`, dette er ikke tatt med i diagrammet for å gjøre det tydeligere å lese.

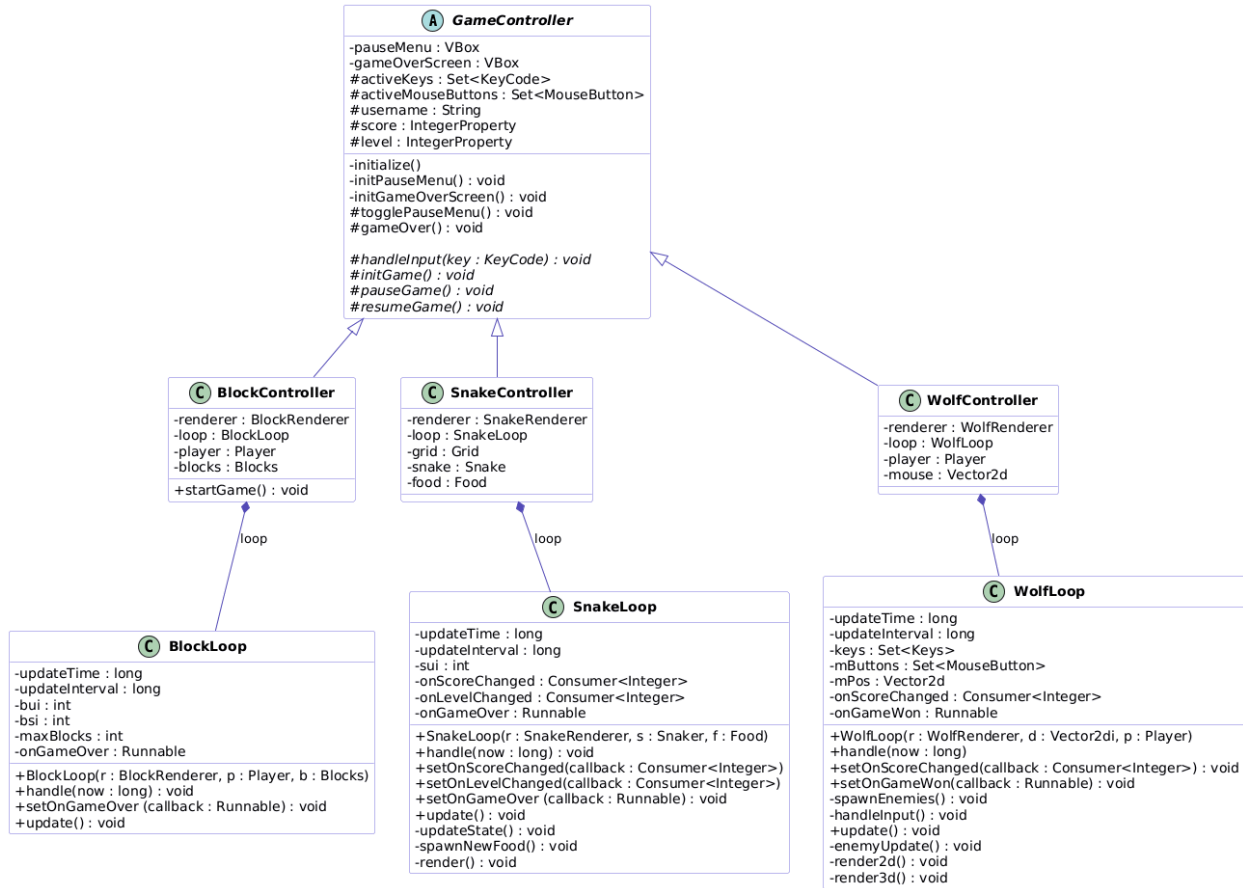


Figure 1: Klassediagram

### 3 Spørsmål

I denne delen skal dere reflektere rundt egne valg, og vise forståelsen dere har for objektorientert programmering og pensum i emnet. Dere skal besvare følgende spørsmål:

1. Hvilke deler av pensum i emnet dekkes i prosjektet, og på hvilken måte? (f.eks. bruk av arv, interface, delegering osv.)

#### Objekter og Klasser

Generelt har vi forsøkt å passe på at tilstanden til programmet til enhver tid er gyldig, vi gjør dette gjennom tilstandsvalidering, og ved å benytte innkapsling der det er hensiktsmessig. Eksempelvis benytter vi private feltvariabler, og get og set metoder de fleste steder i prosjektet.

#### Objektstrukturer

Vi har flere steder i prosjektet koblet sammen flere objekter i logiske objektstrukturer. Dette var en naturlig løsning for oss siden vi lager spill, noe som gjør det logisk å splitte opp funksjonaliteten i f.eks. en spiller-klasse, en fiende-klasse, etc.

Et annet eksempel fra prosjektet er bruken av den unidireksjonelle én-til-én assosiasjonen mellom player-klassen og gun-klassen i wolf spillet. Altså har vi et player-objekt som eier en instans av et gun-objekt som har kontroll på sin egen tilstand som animasjon, om det er lov til å skyte, osv.

## Grensesnitt

Flere steder i prosjektet benytter vi oss av grensesnittene comparable og iterable for å gjøre koden mer leselig. F.eks. bruker vi Iterator flere steder i spillene i for-each løkker, og Comparator i highscoreController for å kunne sortere på henholdsvis Spill, Navn og score i highscore-listen. Vi benytter oss også av de funksjonelle grensesnittene Consumer og Runnable i spillene for å håndtere tilstanden til spillet koblet opp mot brukergrensesnittet. Spesifikt benytter vi de som callback-funksjoner (se delen om designmønstre under).

## Designmønstre

*Delegering* – Dette designmønsteret bruker vi i hele prosjektet for å sikre at hver komponent kun utfører sine egne beregninger og ikke er avhengig av andre komponenter sin interne funksjonalitet. Eksempelvis benytter vi oss av en GameController per spill som kun skal ta seg av koblingen til JavaFX, basic spiller-input, og opprettelse av gameloop og renderer. Gameloopen i hvert spill holder på en referanse til en renderer, og delegerer rendering til denne i stedet for å gjøre alt internt.

Fordelen vi oppnår ved å designe appen på denne måten er at hver Gameloop ikke har noen kunnskap om hvordan renderer håndterer rendering, altså er alle tegne-metoder abstrahert bort fra loopen. Om vi hadde ønsket å bytte metode for å representere spillene våre, f.eks. ved å endre til et annet grafikk-bibliotek, endre fra JavaFX canvas og graphicscontext, e.l., vil vi bare trenge å modifisere renderer-klassen.

*Observatør-Mønsteret* – Som nevnt under Grensesnitt bruker vi de funksjonelle grensesnittene Consumer og Runnable for å passe på at tilstanden i spillet oppdaterer brukergrensesnittet. Dette gjøres ved en liten forenkling av Observatør-Observert-mønsteret. I stedet for å holde en liste over observatører i det observerte objektet, lagrer vi callback-metoder som kalles på når tilstanden oppdateres. (se eksempelkode under)

```
// GameController holder på en IntegerProperty variabel som initialiseres til 0
protected IntegerProperty score = new SimpleIntegerProperty(0);

// I GameLoop lagres en instans av det funksjonelle grensesnittet Consumer
// og vi oppretter en metode for å definere callback-funksjonen
private Consumer<Integer> onScoreChanged = score -> {};
public void setOnScoreChanged(Consumer<Integer> callback) {
    this.onScoreChanged = callback;
}

// Fra GameController-klassen kaller vi på setOnScoreChanged() med følgende
// lambda funksjon som parameter
loop.setOnScoreChanged(s -> score.set(s));
```

## Arv

Ettersom vi ønsket å lage flere forskjellige spill, valgte vi å lage en abstrakt klasse for å definere hva slags funksjonalitet hver enkelt GameController må ha. Vi startet med å lage et GameController-interface, men ettersom mye av funksjonaliteten skulle være lik i alle spillene (input, pause-meny, gameover-meny, username) var en abstrakt klasse den mest logiske løsningen. Dette ga oss fleksibiliteten til å f.eks. la hver enkelt GameController-subklasse definere hvordan input i spillet skal håndteres, samtidig som superklassen sikrer at strukturen, og fellesmetoder opererer helt likt.

## Unntakshåndtering

Vi har benyttet oss av unntakshåndtering i vår `fileManager`-klasse. Denne klassen skal lese og skrive til en `highscores.txt` fil for å oppdatere og evt. overskrive de tidligere satte highscore-ene. Her brukte vi to try-catch-statements. Bruken av try-catch skulle forhindre at programmet

krasjer om det oppstår en feil ved fil-lesing og -skrivning. Selv om slike feil ofte kan oppstå (f.eks. feil i filnavn, fil som ikke kan leses, etc.), så kan applikasjonen fortsette å kjøre. Det er også derfor vi bruker `catch` med `IOException`, slik at disse feilene blir håndtert av Java automatisk, og i tillegg kan dette gi oss en mer spesifikk error-melding. Med `filereader` vil den f.eks. kunne spytte ut en `FileNotFoundException` for å gjøre det lettere å pinpointe feilene. Vi kunne også benyttet oss av en `PrintStackTrace()`, denne ville gitt oss en enda tydeligere indikasjon hvor feilene oppsto og hvordan vi kunne rette de.

**2.** *Dersom deler av pensum ikke er dekket i prosjektet deres, hvordan kunne dere brukt disse delene av pensum i appen?*

Vi har forsøkt å dekke de fleste delene av pensum ved å finne løsninger som følger mange av prinsippene vi har lært i løpet av semesteret. Noen konsepter har vi likevel ikke brukt i like stor grad som vi kunne, f.eks. bruk av `Iterator` og `Comparator`. Disse blir brukt noen steder i koden, men i all hovedsak har vi kun brukt statiske metoder. Vi kunne funnet løsninger der vi implementerer interface-ene, f.eks. ved å lage en `comparator` hjelpeklasse.

I tillegg kunne innkapslingen av koden vært mer konsekvent. I flere av klassene er det felter som kunne hatt strengere innkapsling, men som tidligere i prosjektet ble benyttet på en måte der de måtte være offentlige. Dette hadde vi ikke tid til å fikse, og generelt skal ikke dette være et problem ettersom verdiene de settes til skal bli validert av funksjonen som endrer de.

**3.** *Hvordan forholder koden deres seg til Model-View-Controller-prinsippet?*

Vi har forsøkt å følge designmønsteret så godt som mulig ved å tydelig splitte prosjektet opp i de forskjellige bestanddelene. For hvert spill har vi delt kode-filene opp i mapper som tilsier hvilken del de hører til, og strukturen er lik i alle spillene:

```
spill/
|-- controller/
|   |-- gamecontroller
|   '-- gameloop
|-- model/
|   |-- player
|   |-- enemy
|   '-- world
'-- view/
    '-- renderer
```

Slik vi tolket prinsippet skal `controller` ikke inneholde noe annet enn koblingen mellom det visuelle og den interne logikken til programmet, programmets logikk skal være delt opp i hensiktsmessige klasser under `model`, og `view` representerer det visuelle som ved bruk av JavaFX blir både `.fxml` filene og tegnefunksjoner til JavaFX objekter.

Altså skal all logikk ligge i en av klassene i `model`, men enkelte steder endte vi av enkelhets skyld opp med logikk i `loop`-klassen (f.eks i "block" for å beregne hvilket level man er på).

**4.** *Hvordan har dere gått frem når dere skulle teste appen deres, og hvorfor har dere valgt de testene dere har? Har dere testet alle deler av koden? Hvis ikke, hvordan har dere prioritert hvilke deler som testes og ikke?*

Når vi skulle teste koden ønsket vi å fokusere på de funksjonelle delene av spillene, og vi laget ingen tester for f.eks. JavaFX. Optimalt sett burde vi hatt noen tester av de forskjellige `controller`-klassene, men ettersom disse er tett knyttet til JavaFX ble det vanskelig å teste funksjonaliteten uten denne avhengigheten.

I tillegg har vi merket at det kunne vært greit å teste `FileManager`-klassen nøyere enn det vi har gjort ettersom denne antar en veldig spesifikk struktur på tekstdokumentet som score-en fra hvert spill lagres i. Dette var noe vi dessverre ikke rakk å gjennomføre, men vi har fått testet at generell lesing og skrivning til

fil fungerer, og at scores fra samme spiller overskriver gamle.

Vi valgte å lage et par tester til hvert av spillene hvor vi sjekker at logikken innad i hver klasse stemmer med det forventede utfallet vi prøvde å designe. Prioriteringen ble å teste **spilleren** og generell funksjonalitet i hvert spill. For eksempel testet vi blokkene og player i **block**-spillet, snake og food i **snake**-spillet, og player og gun i **wolf**-spillet. Vi anså disse som noen av de viktigste delene i hvert spill og derfor mente vi at det var de mest relevante å teste.

## 4 KI-Deklarasjon

1. Måtte du bruke KI for å få til prosjektet, eller var det bare tidsbesparende?
2. Dersom du brukte KI, prøvde du flere modeller? Lekte du deg litt med hva mulighetene er?
3. Dersom du brukte KI, i hvilken grad? Skrev den koden din? Spurte du om hjelp med debugging?

*Dersom dere ble inspirerte, gjerne reflekter enda mer rundt bruken av KI i fag som dette, og gjerne dra det videre inn i fremtidig studiehverdag og jobbmarked, er de som studerer software-relaterte studier “cooked”? Eller er KI bare et fantastisk verktøy.*

### Clemens:

Personlig var bruken min av KI for å gjøre samarbeidet litt lettere å forstå og forklare konsept og syntax som jeg ikke helt fikk til. Jeg benyttet meg i hovedsak av Claude i en separat chat i nettleseren for å minimere hvor mye jeg benyttet meg av hjelpen til KI.

Da vi begge hadde påskeferie og jobbet mye fra hverandre ble KI også brukt for å forklare oppsett og format av prosjektet når vi ikke hadde mulighet til å samarbeide og forklare direkte. Da mye av kodingen i utgangspunktet ble skrevet i separate repositories og senere skulle kombineres sammen til et samlet prosjekt, benyttet jeg meg av KI for å hjelpe meg til å ”oversette” koden jeg hadde skrevet inn i repositoriet til appen som Albert hadde laget.

Da vi hadde laget flere ”spill” som skulle bli håndtert av et app interface, brukte vi en større felles controller, som koblet sammen de separate GameControllerne til f.eks ”Block” spiller som jeg lagde. Her trengte jeg da hjelp av KI for å forstå hvordan jeg kunne benytte meg av flere controllere og den separate blockController.

Under selve kodingen av prosjektet har KI blitt brukt så lite som mulig. Den har aldri blitt brukt for å gjøre direkte endringer i koden, men har blitt brukt separat som i Claude chat for å hinte eller dytte meg i rett retning for fullføre mer komplekse deler av kodingen. I hovedsak ble Claude brukt, men hvis jeg bare hadde små spørsmål om hva noe betydde eller jeg hadde brukt opp min daglige Claude bandwidth ble også chatgpt brukt.

### Albert:

Jeg benyttet meg av KI så lite som mulig under arbeidet med prosjektet. I all hovedsak ble KI brukt som et hjelpeverktøy til å feilsøke og forklare konsepter som var utydelige for meg da jeg jobbet.

Eksempelvis var det ganske vanskelig å forstå hvordan JavaFX var satt opp og skulle brukes i starten av prosjektet, så i tillegg til å lese dokumentasjonen brukte jeg Claude til å forklare hvordan de forskjellige JavaFX elementene henger sammen og hva som er god praksis når man benytter seg av de.

I tillegg var KI veldig tidsbesparende når det gjelder feilsøking og tolkning av feilmeldinger. Jeg opplevde ofte at feilmeldingene fra JavaFX var veldig lange og utydelige, så i stedet for å bruke unødvendig tid på å lese nøye gjennom og tolke, var det ofte enklere å lime inn feilmeldingen til Claude og få en forklaring på en

mer menneskeleselig måte.

KI ble også benyttet noe for å hjelpe å identifisere bugs i spillene, som feil i matematikken bak, eller feil i rendering prosessen. Blant annet var det mye til hjelp for å identifisere at en visuell bug kom av hvilken rekkefølge JavaFX renderer streker på en "canvas".

### **Felles:**

Ki blir også brukt for å designe alt av `.fxml` i prosjektet. Vi startet med et enkelt design vi hadde laget selv, men fant fort ut av at vi brukte unødvendig mye tid på å forsøke å gjøre appen fin visuelt. Etter diskusjon med studass og udass kom vi frem til at siden JavaFX og fxml spesifikt ikke er en viktig del av pensum, var det greit å benytte KI til dette. Dette gjorde at vi kunne fokusere tiden vår på funksjonaliteten av appen, og implementere all funksjonaliteten vi ønsket.